

Lezione — Modulo 3B: Gestione dei Pacchetti Software

Corso: Lab Amministrazione di Sistemi T **Materiale:** `software.pdf` — Marco Prandini, DISI UniBo (46 slide) **Prerequisiti:** 0A (filesystem Linux), 2C (tar/archiviazione)

Obiettivo

Al termine di questa lezione Lorenzo deve saper installare, aggiornare, interrogare e rimuovere pacchetti su Debian usando `apt` e `dpkg`, comprendere la struttura dei repository e il meccanismo delle dipendenze.

1. Il Problema della Gestione del Software

Ogni software ha un **ciclo di vita**: installazione → aggiornamento → disinstallazione.

Le **problematiche** che rendono non banale questo ciclo sono: - **Prerequisiti hardware/SO**: il software deve girare sull'architettura giusta - **Dipendenze da altri componenti software**: librerie, tool di sistema - **Configurazione**: ogni software ha parametri specifici

Installazione manuale (perché esiste e perché è scomoda)

Da binari: copia manuale nei percorsi corretti, verifica manuale compatibilità architettura e dipendenze.

Da sorgente (caso tipico in Linux: archivio `.tar.gz`, codice C, build via `autoconf`):

reperimento → estrazione → esame scelte → configurazione sorgenti → compilazione → installazione

Dettaglio del flusso:

```
cd /usr/local/src
tar tvzf net-snmp-5.4.tar.gz      # lista contenuti prima di estrarre
tar xvzf net-snmp-5.4.tar.gz     # estrae
cd net-snmp-5.4/
./configure --help               # esplora opzioni disponibili
./configure [opzioni scelte]     # configura (genera Makefile, produce config.log)
make                             # compila (NON come root)
sudo make install                # installa (qui serve root)
```

Regola di sicurezza: solo `sudo make install` richiede root. Compilare come root è cattiva prassi: esistono casi di malware che sfruttano sorgenti malevoli scaricati ed eseguiti come root.

Autenticità del software scaricato

Prima di estrarre un archivio, verificarne l'autenticità:

```
gpg --verify FILE.asc FILE.tar.gz  # (mostra key id)
gpg --keyserver pgpkeys.mit.edu --recv-key <KEY_ID>
gpg --verify FILE.asc FILE.tar.gz  # ripetere
```

Alternativa con fingerprint:

sha256 -c FILE.sha256

Riflessioni sull'installazione da sorgente

Vantaggi teorici (verifica del codice) che nella pratica spesso non si sfruttano. Svantaggi: - Più difficile da mantenere - Richiede molti componenti ausiliari (header, librerie, compilatori, linker) — che **possono essere vettori di attacco** (cfr. Ken Thompson Hack)

Questo è il motivo per cui le **distribuzioni e i pacchetti** sono nati: - Chiavi di verifica installate una volta per tutte (comodo, ma SPOF) - Gestione automatica delle dipendenze - Garanzia di compatibilità binaria tra tutti gli elementi del set

2. Distribuzioni e Pacchetti

Cos'è un pacchetto

Un file singolo che contiene in forma compatta: - Software precompilato - Criteri per la verifica della compatibilità e dei prerequisiti - Procedure di pre/post-installazione

La garanzia di compatibilità richiede di vincolare tre parametri: - **Architettura** (amd64, i386, arm64, ...) - **Versione della distribuzione** (es. bookworm, jammy, ...) - **Versione del software** contenuto

Formato dei nomi

Debian/Ubuntu → formato .deb:

aptitude-0.2.15.9-2_i386.deb

↑nome ↑ver.sw ↑ver.pkg ↑arch

RedHat/CentOS/Fedora → formato .rpm:

httpd-2.4.6-45.el7.centos.x86_64.rpm

Criteri per la scelta di una distribuzione

Criterio	Descrizione
Architetture supportate	Quasi tutte supportano amd64; per architetture esotiche verificare
Stabilità vs aggiornamento	Distro “stabili” includono solo sw collaudato; altre preferiscono novità anche a costo di minor robustezza
Versioned vs rolling	Versioned: aggiornamenti correttivi solo durante il ciclo; rolling: sempre all'ultima versione
LTS	Per server: supporto 5/7 anni con solo security backport
Ampiezza pacchetti	Da 1500 (minimali) a 26000+ (Debian)

Debian (la nostra VM): versioned, orientata alla stabilità, ~26000 pacchetti.

3. Due Famiglie: Debian e Red Hat

Quasi tutte le distro Linux derivano da una di queste due.

Entrambe usano una struttura a tre livelli: 1. **Tool di basso livello** per gestire singoli pacchetti (dpkg / rpm) 2. **Tool intermedi** per gestione coordinata di pacchetti e dipendenze (apt / yum/dnf) 3. **Tool per il reperimento automatico** da repository

4. Dipendenze: il Grafo

Il concetto centrale è il **grafo delle dipendenze**: $A \rightarrow B$ significa che A “serve” per B.

Le dipendenze possono essere: - **Logiche**: non ha senso avere un linguaggio server-side senza un web server (es. php $\rightarrow$ apache) - **Fisiche**: un binario linkato dinamicamente non gira senza le librerie che importa (es. firefox $\rightarrow$ gtk $\rightarrow$ glibc)

Il package manager conosce l'intero grafo e risolve automaticamente le dipendenze al momento dell'installazione e della rimozione.

Pacchetti base e pacchetti -dev

Per ogni libreria esistono due pacchetti distinti:

Pacchetto	Contiene	Usato per
zlib1g	/usr/lib/libz.so.1.2.3.3 (linking dinamico)	Eseguire programmi che usano zlib
zlib1g-dev	/usr/lib/libz.a, /usr/include/zlib.h, ... (linking statico + header)	Compilare programmi che usano zlib

Su sistemi deb: suffisso -dev. Su sistemi rpm: suffisso -devel.

Risparmio di spazio: i sistemi non usati per sviluppo non installano i -dev.

Verificare le dipendenze dinamiche di un binario

```
ldd /usr/sbin/sshd
# output: lista di librerie .so con path e indirizzo in memoria
# es: libz.so.1 => /usr/lib/libz.so.1 (0xb7d63000)
```

5. Repository

I pacchetti vengono distribuiti tramite **repository** (repo): raccolte indicizzate di pacchetti, online o su filesystem locali.

Il package manager legge per ogni repo l'indice e i metadati: - conosce quali versioni sono disponibili
- conosce le dipendenze tra pacchetti

Configurazione repo in Debian/apt

File principale: `/etc/apt/sources.list` Frammenti aggiuntivi: `/etc/apt/sources.list.d/*.list`

Formato di una riga:

```
deb http://archive.ubuntu.com/ubuntu bionic-updates universe
```

Per aggiungere un repository di terze parti (es. VirtualBox):

```
# Creare il file:
# /etc/apt/sources.list.d/virtualbox.list
deb http://download.virtualbox.org/virtualbox/debian xenial contrib
```

Il database locale di apt/dpkg è in `/var/lib/dpkg/` e `/var/lib/apt/`.

6. Comandi dpkg e apt su Debian

Tabella comandi principali

Operazione	Comando dpkg	Comando apt
Update indice repo	—	<code>apt update</code>
Cercare pacchetto	—	<code>apt search <keyword></code>
Installare da repo	—	<code>apt install <nome></code>
Installare da file .deb	<code>dpkg -i file.deb</code>	—
Aggiornare tutto	—	<code>apt upgrade</code>
Aggiornare un pacchetto	—	<code>apt upgrade <nome></code>
Rimuovere (mantieni config)	<code>dpkg -r <nome></code>	<code>apt remove <nome></code>
Rimuovere (+ config)	<code>dpkg -P <nome></code>	<code>apt purge <nome></code>
Rimuovere dipendenze orfane	—	<code>apt autoremove</code>
Info pacchetto	<code>dpkg -s <nome></code>	<code>apt show <nome></code>
Lista file installati	<code>dpkg -L <nome></code>	—
A quale pkg appartiene un file	<code>dpkg -S /path/file</code>	—
Lista pacchetti installati	<code>dpkg -l</code>	<code>apt list --installed</code>
Lista aggiornabili	—	<code>apt list --upgradable</code>
Provenienza pacchetto	—	<code>apt-cache showpkg <nome></code>

Nota: `apt-key` per la gestione delle chiavi GPG è **deprecato**. Le chiavi GPG dei repo vanno in `/etc/apt/trusted.gpg.d/`.

apt remove vs apt purge

- `apt remove`: rimuove i binari, **mantiene** i file di configurazione in `/etc/`
 - `apt purge`: rimuove tutto inclusi i file di configurazione
 - `apt autoremove`: rimuove i pacchetti installati come dipendenze che non servono più
-

7. Autenticità dei Pacchetti (sistema distribuzioni)

Nelle distribuzioni la firma è gestita centralmente: - I *maintainer* distribuiscono le chiavi di verifica nei media di installazione o nei repo - I set di chiavi sono gestiti con GnuPG - I keyring `.deb` vanno in `/etc/apt/trusted.gpg.d/`

```
# Aggiungere la chiave pubblica di un repo (metodo moderno):  
gpg -o /etc/apt/trusted.gpg.d/mio-repo.gpg --dearmor pubkey.txt
```

8. Gestire la Provenienza dei Pacchetti (Software Injection)

Rischio: se un repo di terze parti contiene un pacchetto con lo stesso nome di un pacchetto di sistema, dichiarandolo a versione più recente, il package manager lo installerà al posto di quello ufficiale → **software injection**.

Per controllare e limitare questo rischio:

```
# Verificare da quale repo proviene un pacchetto installato:  
apt-cache showpkg <nome>
```

Version locking/pinning (bloccare un pacchetto a una versione specifica): - Editare `/etc/apt/preferences.d/*` - Documentazione: <https://wiki.debian.org/AptPreferences>

9. Problematiche di Aggiornamento e Disinstallazione

Aggiornare un pacchetto può causare problemi su: 1. **Prerequisiti:** i pacchetti-dipendenza potrebbero dover essere aggiornati, ma questo potrebbe rompere altri software che li usano 2. **Configurazione:** nuove versioni possono introdurre format incompatibili con le config già presenti 3. **Dipendenze di altri software:** modifiche alle interfacce possono rompere software terzi 4. **PATH:** l'ordine di ricerca degli eseguibili può selezionare la versione sbagliata 5. **Loader dinamico:** `/etc/ld.so.conf` controlla quali librerie `.so` vengono caricate; `ldconfig` aggiorna la cache

```
# Verificare quale libreria viene caricata effettivamente:  
ldd /usr/sbin/sshd | grep libz
```

```
# Forzare una versione alternativa:  
export LD_LIBRARY_PATH=/usr/local/lib  
ldd /usr/sbin/sshd | grep libz    # ora punta a /usr/local/lib/libz.so.1
```

Disinstallazione: presenta gli stessi rischi dell'aggiornamento. Il *grafo delle dipendenze* è il valore aggiunto principale del sistema a pacchetti: rende prevedibili gli effetti a cascata.

10. Panoramica: Snap/Flatpak e Container

Il PDF introduce questi concetti come evoluzione del problema della gestione del software.

Snap/Flatpak: pacchetti indipendenti dalla distribuzione, con tutte le dipendenze incluse. Utili per software particolarmente longevo o basato su funzioni non standard.

Virtual Environments (VE) / Container: invece di isolare solo il pacchetto, si isola l'intero *ambiente di esecuzione* di un processo. Il kernel Linux fornisce: - **cgroups:** misurano e limitano le risorse (CPU, RAM, I/O, rete) - **namespaces:** ogni processo vede istanze virtuali delle risorse (mnt, pid, net, ipc, uts, user) - **union-capable filesystems:** combinano layer di filesystem (base dei container)

I container (Docker, LXC) combinano questi tre meccanismi.

Esercizi Guidati sulla VM Debian

Prima di iniziare:

```
cd ~/sysAdmin-lab && vagrant up && vagrant ssh
```

Es. 1 — Esplorazione base

```
# Aggiorna l'indice dei repository
sudo apt update

# Quanti pacchetti sono aggiornabili?
apt list --upgradable 2>/dev/null | wc -l

# Quanti pacchetti sono installati?
dpkg -l | wc -l
```

Es. 2 — Installare un pacchetto e ispezionarlo

```
# Installa tree (visualizza filesystem ad albero)
sudo apt install tree

# Dove sono stati installati i suoi file?
dpkg -L tree

# Cosa fa esattamente?
apt show tree

# Prova:
tree /etc/systemd/system/
```

Es. 3 — Cercare e ispezionare prima di installare

```
# Cerca pacchetti correlati a htop
apt search htop

# Vedi i metadati (dipendenze, dimensione, descrizione)
apt show htop

# Installa
sudo apt install htop

# Verifica da quale repository viene
apt-cache showpkg htop
```

Es. 4 — Risalire dal file al pacchetto

```
# A quale pacchetto appartiene /bin/ls?
dpkg -S /bin/ls

# A quale pacchetto appartiene /usr/bin/ssh?
dpkg -S /usr/bin/ssh

# Quali file ha installato quel pacchetto?
dpkg -L openssh-client
```

Es. 5 — Dipendenze dinamiche con ldd

```
# Quali librerie usa sshd?
ldd /usr/sbin/sshd

# Conta quante librerie dinamiche usa
ldd /usr/sbin/sshd | wc -l

# Confronta con un binario più semplice
ldd /bin/ls
```

Es. 6 — Rimozione

```
# Rimuovi tree mantenendo config (se esistesse)
sudo apt remove tree

# Verifica che sia andato via
which tree    # deve dare errore

# Rimuovi le dipendenze orfane
sudo apt autoremove
```

Es. 7 — Esplora `/etc/apt/sources.list`

```
cat /etc/apt/sources.list
ls /etc/apt/sources.list.d/
ls /etc/apt/trusted.gpg.d/
```

Connessioni

- **Con il modulo precedente (3A — systemd)**: i servizi avviati da systemd sono programmi installati tramite pacchetti. `apt install nginx` installa il pacchetto e crea automaticamente la unit systemd; `dpkg -L nginx` mostra dove è finito il file unit.
 - **Con Security (S1 — Enumerazione)**: Nmap identifica i servizi in ascolto; ogni servizio è un pacchetto installato. Conoscere `dpkg -l` e `apt list --installed` è fondamentale per l'inventario di superficie d'attacco. La slide sulla *software injection* è direttamente rilevante per supply chain attacks.
-

Riepilogo

- Un **pacchetto** è un file che contiene binari precompilati + metadati (dipendenze, versione, architettura) + script di pre/post-installazione. Debian usa `.deb`; RedHat usa `.rpm`.
- Il **repository** è una raccolta indicizzata di pacchetti; `apt update` sincronizza l'indice locale. La lista dei repo è in `/etc/apt/sources.list` e `/etc/apt/sources.list.d/`.
- `dpkg` opera sul singolo file `.deb`; `apt` gestisce il ciclo completo incluse le dipendenze automaticamente. I comandi critici: `apt install/remove/purge/autoremove/upgrade`, `dpkg -L/-S/-l`.
- Il **grafo delle dipendenze** è il valore aggiunto principale: rende prevedibile cosa si installa, cosa si rompe all'aggiornamento, cosa rimane orfano alla rimozione.
- L'**autenticità** è garantita da firme GPG: le chiavi di verifica dei repo vanno in `/etc/apt/trusted.gpg.d/`. Un repo non firmato è un rischio di *software injection*.

Collegati

- [[Appunti_modulo 3B]]
- [[appunti_modulo3B_gestione_pacchetti]]

Hub: [[master_map_studio]] · [[glossario_sysadm]] · [[concept_maps]] · [[troubleshooting_vm]]
· [[metodo_studio_esami_pratici]]